UNIVERSIDAD DE SAN BUENAVENTURA
Tecnología en Desarrollo de Software · VI semestre
Programación Avanzada y Aplicada

Historial SQLite: operaciones CRUD

Crear, consultar, actualizar y eliminar con evidencia

Proyecto final: **AsistenteIA**, asistente académico móvil con .NET MAUI y una API REST de IA

Autor: Jeferson Wilderman González Tenjo y equipo
Docente: Mg. Sandra Esperanza Sánchez Barragán
Fecha: 16 de octubre de 2026
Versión evaluada: rama `feat/rediseno-interfaz` (commit 6a15fc1)

1. Introducción

AsistenteIA guarda localmente, en una base SQLite, las conversaciones que el usuario decide conservar y las plantillas de prompts. Este documento muestra el esquema, el código de cada operación CRUD y la evidencia de que funcionan en la aplicación.

2. Esquema

```
CREATE TABLE ConversationRecord (
  Id INTEGER PRIMARY KEY AUTOINCREMENT,
  Prompt TEXT, Answer TEXT, CreatedAt BIGINT,
  Category TEXT, IsVerified INTEGER, Model TEXT
);
CREATE INDEX ConversationRecord_Category ON ConversationRecord(Category);

CREATE TABLE PromptTemplate (
  Id INTEGER PRIMARY KEY AUTOINCREMENT,
  Name TEXT, Category TEXT, Text TEXT
);
```

Elemento	Detalle
Ubicación	FileSystem.AppDataDirectory/asistenteia.db3 (en Windows sin empaquetar: %LOCALAPPDATA%\User Name\co.edu.usb.asistenteia\Data).
Inicialización	Perezosa y segura entre hilos (SemaphoreSlim) en el primer acceso: crea tablas, índice y seis plantillas iniciales.
Entidad	ConversationRecord (Models/ConversationRecord.cs) con [PrimaryKey, AutoIncrement] e [Indexed] en Category.

3. Código de las operaciones (DatabaseService)

```
// ---- Historial (CRUD) ----

public async Task<int> SaveAsync(ConversationRecord record)
{
    var db = await GetConnectionAsync();
    return record.Id == 0 ? await db.InsertAsync(record) : await db.UpdateAsync(record);
}

public async Task<List<ConversationRecord>> GetConversationsAsync(string? search = null, string? category = null)
{
    var db = await GetConnectionAsync();
    var query = db.Table<ConversationRecord>();
    if (!string.IsNullOrWhiteSpace(category))
        query = query.Where(r => r.Category == category);
    if (!string.IsNullOrWhiteSpace(search))
    {
        string s = search.Trim();
        query = query.Where(r => r.Prompt.Contains(s) || r.Answer.Contains(s));
    }
    return await query.OrderByDescending(r => r.CreatedAt).ToListAsync();
}

public async Task<ConversationRecord?> GetConversationAsync(int id)
{
    var db = await GetConnectionAsync();
    return await db.Table<ConversationRecord>().Where(r => r.Id == id).FirstOrDefaultAsync();
}

public async Task<int> DeleteAsync(ConversationRecord record)
{
    var db = await GetConnectionAsync();
    return await db.DeleteAsync(record);
}
```

4. Matriz CRUD

Operación	Método	Dónde se usa	Evidencia
Crear	SaveAsync → InsertAsync	Botón Guardar de una respuesta (ChatViewModel.SaveAsync)	P07a, P07c
Consultar	GetConversationsAsync (búsqueda, filtro, orden por fecha) y GetConversationAsync(id)	Historial y Detalle	P07b, A11, P09
Actualizar	SaveAsync → UpdateAsync	Marcar como verificada y cambiar categoría (DetailViewModel)	A10
Eliminar	DeleteAsync	Papelera del historial o botón Eliminar del detalle, con confirmación	P08

5. Evidencias

Crear (P07)

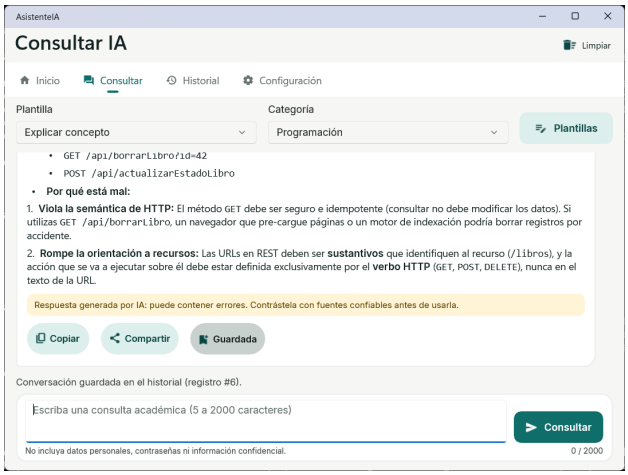


Figura 1. Guardar: "registro #6".

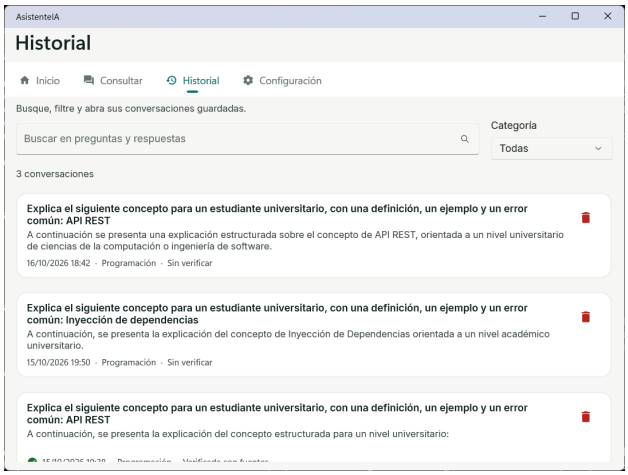


Figura 2. El registro aparece en el historial.

P07 · Tabla ConversationRecord en SQLite

```
$ sqlite3 asistenteia.db3 "select Id, Prompt, Category, IsVerified, Model from ConversationRecord"

  1 | ...ón, un ejemplo y un error común: API REST | Programación | 1 | gemini-3.8-flash
  5 | ...un error común: Inyección de dependencias | Programación | 0 | gemini-3.8-flash
  6 | ...ón, un ejemplo y un error común: API REST | Programación | 0 | gemini-3.8-flash

3 registros. El último corresponde a la consulta de P01 guardada en P07.
```

Figura 3. Registro en la tabla ConversationRecord consultado directamente en SQLite.

Consultar: búsqueda, filtro y persistencia (A11, P09)

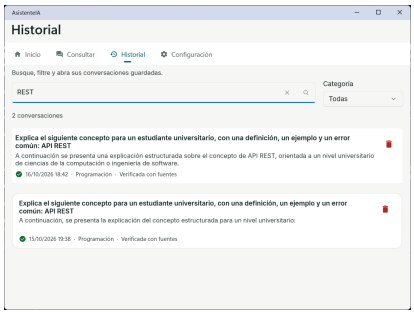


Figura 4. Búsqueda "REST".

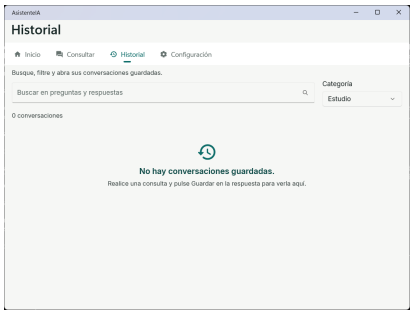


Figura 5. Filtro por categoría.

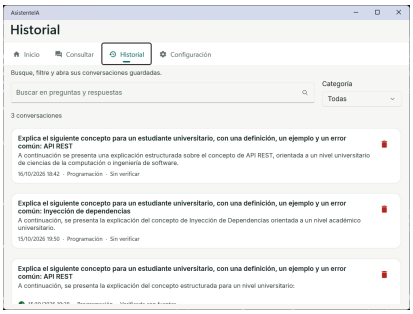


Figura 6. Persiste tras reiniciar.

Actualizar (A10)

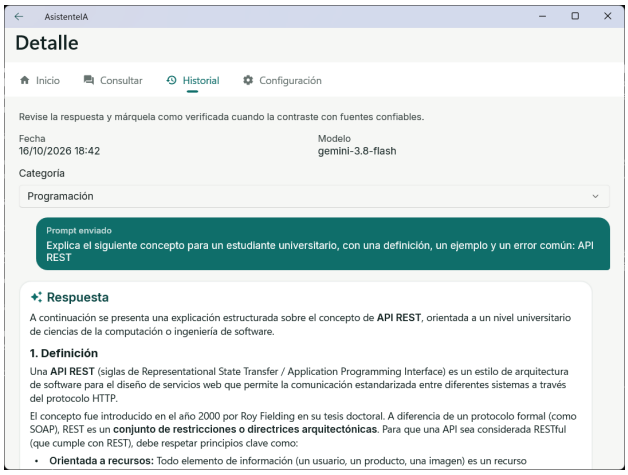


Figura 7. Antes: sin verificar.

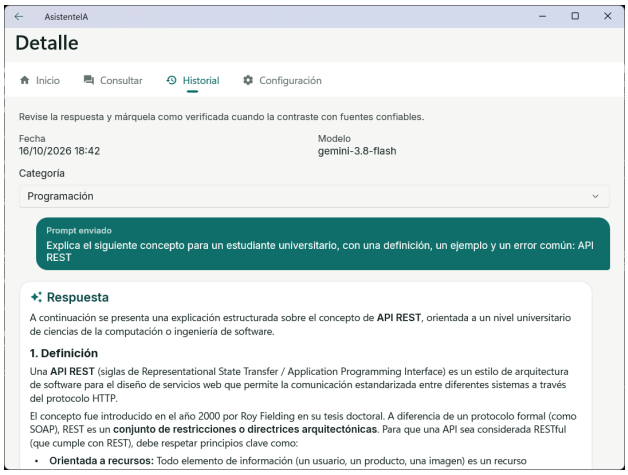


Figura 8. Después: verificada (UpdateAsync).

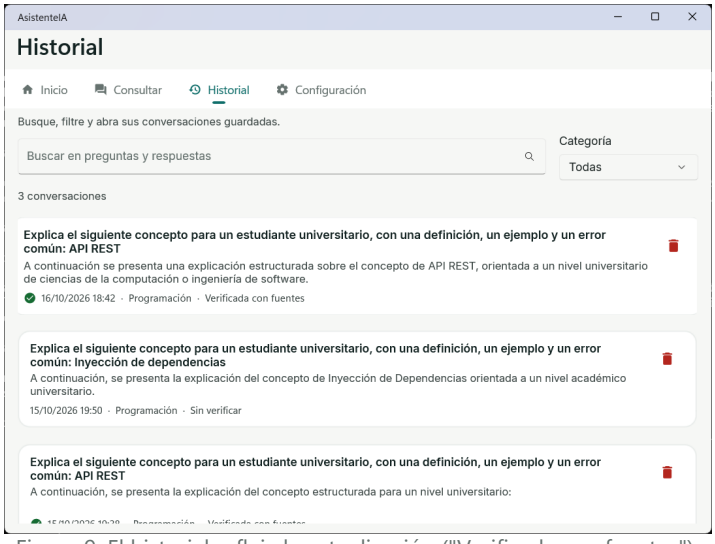


Figura 9. El historial refleja la actualización ("Verificada con fuentes").

Eliminar (P08)

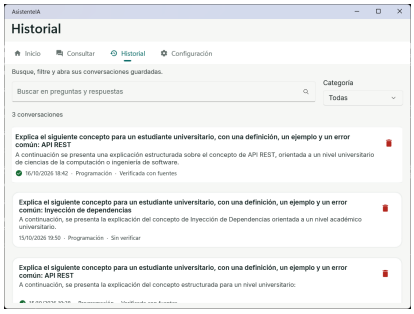


Figura 10. Antes: 3.

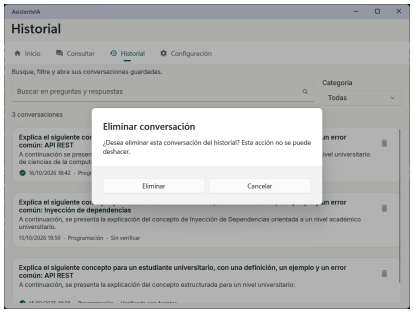


Figura 11. Confirmación.

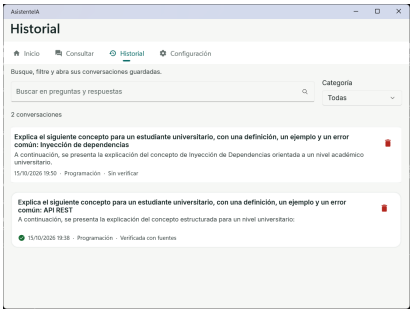


Figura 12. Después: 2.

CRUD de plantillas (A01–A03)

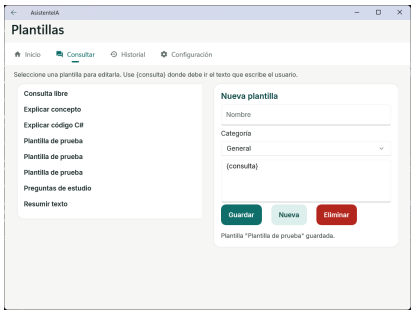


Figura 13. Crear.

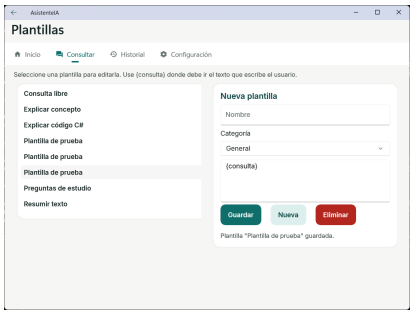


Figura 14. Editar.

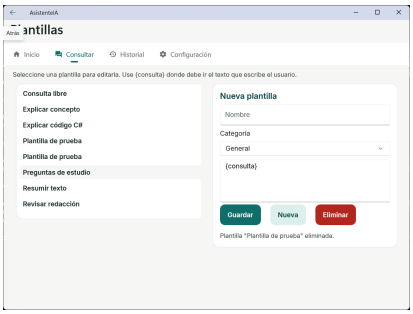


Figura 15. Eliminar.

6. Conclusión

Las cuatro operaciones CRUD sobre el historial están implementadas en DatabaseService, se usan desde los ViewModels y quedaron verificadas en la aplicación con evidencia, incluida la consulta directa a la base SQLite y la persistencia después de reiniciar.